La Citadelle

Activer le protocole de sécurité

Les patrouilles sont opérationnelles.

Il reste une dernière tâche avant que Turing valide votre passage : le système de sécurité doit savoir réagir lorsqu'une donnée est impossible ou dangereuse.

> « Une erreur ignorée devient tôt ou tard une brèche. »

Vous allez découvrir :

- `throw` ;
- `try` ;
- `catch` ;
- la gestion explicite des erreurs.

Cette mission réutilise également les conditions et les boucles.

Instructions

Complétez les fonctions du fichier `securite.js`.

1. Valider un niveau d'énergie

Implémentez :

```
validateEnergy(energy)
```

Règles :

- si `energy` est inférieur à `0`, lancer une erreur avec :

```
throw new Error("Énergie invalide");
```

- sinon, retourner la valeur `energy`.

Exemples :

```
validateEnergy(50);
// => 50
```

```
validateEnergy(-1);
// => lance une erreur
```

2. Sécuriser la validation

Implémentez :

```
safeEnergyCheck(energy)
```

La fonction doit appeler `validateEnergy(energy)` dans un bloc `try`.

Si tout va bien, elle retourne :

```
Énergie valide
```

Si une erreur est levée, interceptez-la avec `catch` et retournez :

```
Énergie invalide
```

3. Valider un code de porte

Implémentez :

```
validateGateCode(code)
```

Règles :

- `"ALPHA"` → retourner `"ALPHA"`
- `"BETA"` → retourner `"BETA"`
- `"OMEGA"` → retourner `"OMEGA"`
- tout autre code → lancer :

```
throw new Error("Code inconnu");
```

Utilisez `switch`.

4. Vérifier un code sans faire planter le programme

Implémentez :

```
safeGateCheck(code)
```

La fonction appelle `validateGateCode(code)`.

- si le code est valide, retourner `"Code accepté"`;
- si une erreur est levée, retourner `"Code refusé"`.

Utilisez `try` / `catch`.

5. Vérifier plusieurs tentatives

Implémentez :

```
countValidAttempts(text)
```

Chaque caractère du texte représente une tentative :

- `"A"` est valide ;
- `"B"` est valide ;
- les autres caractères sont invalides.

Parcourez `text` avec `for...of` et comptez le nombre de tentatives valides.

Exemple :

```
countValidAttempts("ABXABA");
// => 5
```

6. Compter avant verrouillage

Implémentez :

```
attemptsBeforeLock(sequence)
```

Chaque caractère représente une tentative :

- `"V"` = tentative valide ;
- `"X"` = tentative invalide.

Le système se verrouille dès qu'il rencontre **3 erreurs**.

Parcourez la chaîne dans l'ordre et retournez le nombre total de tentatives traitées avant le verrouillage inclus.

Exemple :

```
attemptsBeforeLock("VXVXXV");
// => 5
```

Les trois `X` sont rencontrés aux positions 2, 4 et 5.

Utilisez une boucle et une condition.

À rechercher si nécessaire

- `JavaScript throw Error`
- `JavaScript try catch`
- `JavaScript switch`
- `JavaScript for of`

▶ Aides